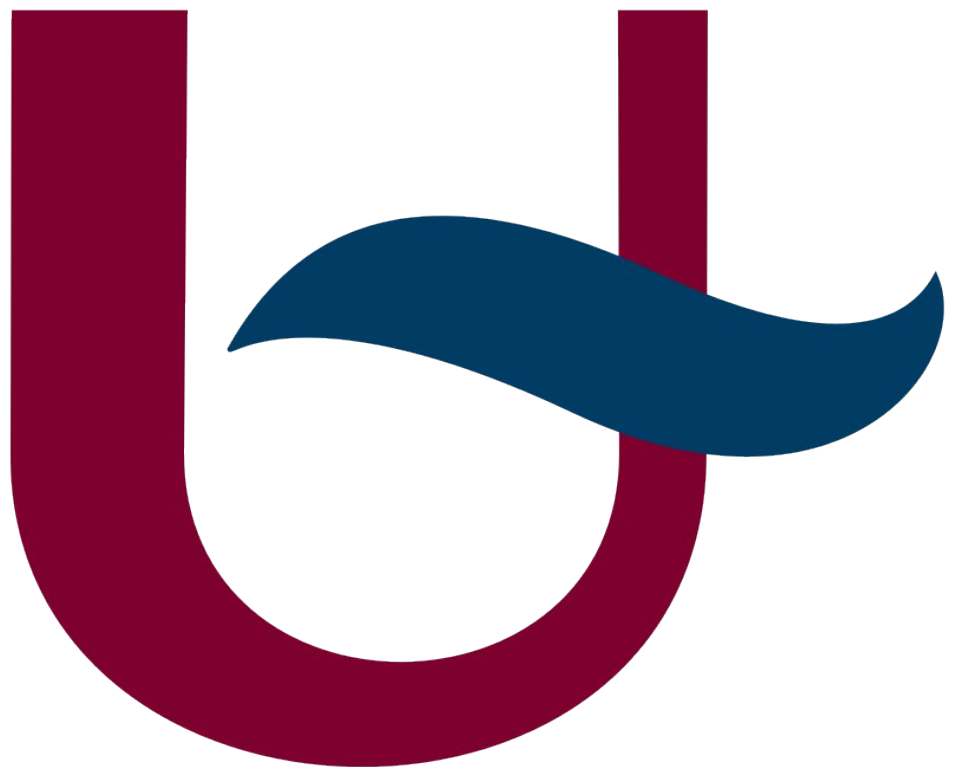


I-MAP: LOGBOOK (AI USAGE)

TSMK



LLM request

△ [Claude](#) was used for this prompt.

Could help me add json and csv logging to my sd-card logging file:

Here is the data:

```
{ "timestamp":481444,"date":160425,"time":8270680,"t_llc":30.18518829345703
,"t_hlc":22.75,"t_bp":17.5,"t_ss":15.25,"t_bm_1":25.5,"t_bm_2":0,"t_bm_3":0,"t_
bm_4":0,"t_esc_1":27.25,"t_esc_2":0,"t_esc_3":0,"t_esc_4":0,"i_ss":0.0536666
6615009308,"i_llc":0.016368335112929344,"i_hlc":0.15348666906356812,"roll
":2.058988094329834,"pitch":-
2.4174697399139404,"yaw":159.93450927734375,"w":0.17376708984375,"i":-
0.01763916015625,"j":-
0.0213623046875,"k":0.984375,"rpm_wheel_1":0,"rpm_wheel_2":0,"rpm_wheel
_3":0,"rpm_wheel_4":0,"rpm_bm_1":0,"rpm_bm_2":0,"rpm_bm_3":0,"rpm_bm_
4":0,"latitude":51.08921813964844,"longitude":4.243618965148926,"velocity":0
,"altitude":0,"v_cell_1":0.00839197263121605,"v_cell_2":0.0079061090946197
51,"v_cell_3":0.008308651857078075,"v_cell_4":0.006697171367704868,"v_b
att":0.031303904950618744,"v_llc":3.292675495147705,"v_hlc":0.0070976610
29547453,"v_phrp_3v3":3.287858486175537,"ch_1":1500,"ch_2":1528,"ch_3":
1008,"ch_4":1468,"ch_5":1038,"ch_6":1065,"ch_7":1000,"ch_8":1000,"ch_9":15
00,"ch_10":2000,"ch_11":1500,"ch_12":1500,"ch_13":1500,"ch_14":1990,"pwm
_bm_1":1528,"pwm_bm_2":0,"pwm_bm_3":0,"pwm_bm_4":0,"pwm_ss":1468,"c
_s":12,"failsafe":0,"dead_switch":0,"i_esc_1":0,"i_esc_2":0,"i_esc_3":0,"i_esc_4
":0}
```

LLM response

```
#define CHIP_SELECT BUILTIN_SDCARD
String _binFilename;
File _binFile;
String _csvFilename;
File _csvFile;
String _jsonFilename;
File _jsonFile;
bool _jsonFirstRecord;

bool _jsonFirstRecord; // tracks whether to prepend a comma

bool SDCard::init() {
    interrupts_enable();
    if (!SD.begin(CHIP_SELECT)) {
        setError(ModuleError::INIT_FAILED);
        return false;
    }

    String filename = generateUniqueFilename(".csv");
    _binFilename = filename + ".bin";
    _csvFilename = filename + ".csv";
    _jsonFilename = filename + ".json";

    interrupts_disable();
    _binFile = SD.open(_binFilename.c_str(), FILE_WRITE);
    _csvFile = SD.open(_csvFilename.c_str(), FILE_WRITE);
    _jsonFile = SD.open(_jsonFilename.c_str(), FILE_WRITE);

    if (!_csvFile || !_jsonFile) {
        setError(ModuleError::INIT_FAILED);
        interrupts_enable();
        return false;
    }

    // CSV header
    _csvFile.println(
        "timestamp,date,time,latitude,longitude,velocity,altitude,c_s,"
        "ch_1,ch_2,ch_3,ch_4,ch_5,ch_6,ch_7,ch_8,ch_9,ch_10,failsafe,"
        "rpm_wheel_1,rpm_wheel_2,rpm_wheel_3,rpm_wheel_4,"
        "v_cell_1,v_cell_2,v_cell_3,v_cell_4,v_rail_3V3,v_rail_5v0,"
        "i_esc_1,i_esc_2,i_esc_3,i_esc_4,i_etc,i_llc,i_hlc,"
        "t_llc,t_hlc,t_bp,t_ss,t_bm_1,t_bm_2,t_bm_3,t_bm_4,"
        "t_esc_1,t_esc_2,t_esc_3,t_esc_4,"
        "roll,pitch,yaw,w,i,j,k"
```

```
);
_csvFile.flush();

// JSON: open the array
_jsonFile.print('[');
_jsonFile.flush();
_jsonFirstRecord = true;

interrupts_enable();
return true;
}

// — JSON helper: write one key:value pair
// Avoids any String heap allocation – writes directly to the file.

static void jInt(File &f, const char *key, long val, bool comma =
true) {
    if (comma) f.print(',');
    f.print('"'); f.print(key); f.print("\":"); f.print(val);
}
static void jFloat(File &f, const char *key, double val, uint8_t
decimals, bool comma = true) {
    if (comma) f.print(',');
    f.print('"'); f.print(key); f.print("\":"); f.print(val,
decimals);
}

void SDCard::writeJson(const logging_data_t &d) {
    if (!_jsonFile) return;

    // Separator between records
    if (!_jsonFirstRecord) _jsonFile.print(',');
    _jsonFirstRecord = false;

    _jsonFile.print('{');

    // Timestamp & GPS (first field: no leading comma)
    jInt (_jsonFile, "timestamp", d.timestamp, false);
    jInt (_jsonFile, "date", d.date);
    jInt (_jsonFile, "time", d.time);
    jFloat(_jsonFile, "latitude", d.latitude, 6);
    jFloat(_jsonFile, "longitude", d.longitude, 6);
    jFloat(_jsonFile, "velocity", d.velocity, 3);
    jFloat(_jsonFile, "altitude", d.altitude, 3);
    jInt (_jsonFile, "c_s", d.c_s);

    // RC channels
```

```
jInt(_jsonFile, "ch_1", d.ch_1);
jInt(_jsonFile, "ch_2", d.ch_2);
jInt(_jsonFile, "ch_3", d.ch_3);
jInt(_jsonFile, "ch_4", d.ch_4);
jInt(_jsonFile, "ch_5", d.ch_5);
jInt(_jsonFile, "ch_6", d.ch_6);
jInt(_jsonFile, "ch_7", d.ch_7);
jInt(_jsonFile, "ch_8", d.ch_8);
jInt(_jsonFile, "ch_9", d.ch_9);
jInt(_jsonFile, "ch_10", d.ch_10);
jInt(_jsonFile, "failsafe", d.failsafe);

// RPM
jFloat(_jsonFile, "rpm_wheel_1", d.rpm_wheel_1, 2);
jFloat(_jsonFile, "rpm_wheel_2", d.rpm_wheel_2, 2);
jFloat(_jsonFile, "rpm_wheel_3", d.rpm_wheel_3, 2);
jFloat(_jsonFile, "rpm_wheel_4", d.rpm_wheel_4, 2);

// Voltage
jFloat(_jsonFile, "v_cell_1", d.v_cell_1, 3);
jFloat(_jsonFile, "v_cell_2", d.v_cell_2, 3);
jFloat(_jsonFile, "v_cell_3", d.v_cell_3, 3);
jFloat(_jsonFile, "v_cell_4", d.v_cell_4, 3);
jFloat(_jsonFile, "v_rail_3V3", d.v_rail_3V3, 3);
jFloat(_jsonFile, "v_rail_5v0", d.v_rail_5v0, 3);

// Current
jInt (_jsonFile, "i_esc_1", d.i_esc_1);
jInt (_jsonFile, "i_esc_2", d.i_esc_2);
jInt (_jsonFile, "i_esc_3", d.i_esc_3);
jInt (_jsonFile, "i_esc_4", d.i_esc_4);
jFloat(_jsonFile, "i_etc", d.i_etc, 3);
jFloat(_jsonFile, "i_llc", d.i_llc, 3);
jFloat(_jsonFile, "i_hlc", d.i_hlc, 3);

// Temperature
jFloat(_jsonFile, "t_llc", d.t_llc, 2);
jFloat(_jsonFile, "t_hlc", d.t_hlc, 2);
jFloat(_jsonFile, "t_bp", d.t_bp, 2);
jFloat(_jsonFile, "t_ss", d.t_ss, 2);
jFloat(_jsonFile, "t_bm_1", d.t_bm_1, 2);
jFloat(_jsonFile, "t_bm_2", d.t_bm_2, 2);
jFloat(_jsonFile, "t_bm_3", d.t_bm_3, 2);
jFloat(_jsonFile, "t_bm_4", d.t_bm_4, 2);
jFloat(_jsonFile, "t_esc_1", d.t_esc_1, 2);
jFloat(_jsonFile, "t_esc_2", d.t_esc_2, 2);
jFloat(_jsonFile, "t_esc_3", d.t_esc_3, 2);
jFloat(_jsonFile, "t_esc_4", d.t_esc_4, 2);
```

```
// IMU
jFloat(_jsonFile, "roll", d.roll, 4);
jFloat(_jsonFile, "pitch", d.pitch, 4);
jFloat(_jsonFile, "yaw", d.yaw, 4);
jFloat(_jsonFile, "w", d.w, 6);
jFloat(_jsonFile, "i", d.i, 6);
jFloat(_jsonFile, "j", d.j, 6);
jFloat(_jsonFile, "k", d.k, 6);

_jsonFile.print('}');
_jsonFile.flush();
}

void SDCard::sendData(const logging_data_t log_data) {
    if (!_binFile || !_csvFile || !_jsonFile) {
        setError(ModuleError::LOG_FAILED);
        return;
    }
    setError(ModuleError::OK);

    // — Binary

    uint8_t buffer[sizeof(log_data)];
    memcpy(buffer, &log_data, sizeof(log_data));
    _binFile.write(buffer, sizeof(buffer));
    _binFile.flush();

    // — CSV (unchanged)

    // ... your existing CSV prints ...

    // — JSON

    writeJson(log_data);
}

void SDCard::closeFile() {
    if (_jsonFile) {
        _jsonFile.print(']'); // close the JSON array
        _jsonFile.flush();
        _jsonFile.close();
    }
    if (_csvFile) {
        _csvFile.flush();
        _csvFile.close();
    }
    if (_binFile) {
```

```
        _binFile.flush();  
        _binFile.close();  
    }  
}
```